

RECORD

Full Stack Development - Lab Record

Exercise 1: Configuring the Development Environment and Implementing Basic JavaScript Programs

Exercise 1a: Development Environment Setup

Aim

To configure the development environment by installing Node.js, npm, and VS Code for full-stack web development.

Procedure

1. Download and install Node.js (LTS version) from the official website (<https://nodejs.org>), which includes npm (Node Package Manager).
2. Verify the installation by running `node -v` and `npm -v` in the terminal to check the installed versions.
3. Install Visual Studio Code (VS Code) from <https://code.visualstudio.com> as the primary code editor.
4. Install essential VS Code extensions: Live Server, Prettier, ESLint, and JavaScript (ES6) Code Snippets for enhanced development workflow.
5. Create a test JavaScript file (`test.js`) and run it using `node test.js` in the terminal to confirm the environment is working correctly.

Result

The development environment was successfully configured with Node.js, npm, and VS Code installed and verified.

All tools were tested and confirmed to be functioning correctly for full-stack web development.

Exercise 1b: Interactive Webpage using HTML, CSS and JavaScript

Aim

To create an interactive webpage that demonstrates the use of HTML for structure, CSS for styling, and JavaScript for dynamic user interaction.

Procedure

1. Create the HTML file (`index.html`) with a header, an input field, a display button, color-change buttons, and a counter section.
2. Create the CSS file (`style.css`) to style the page with a gradient header, centered layout, styled buttons, and smooth transitions.
3. Write JavaScript (`script.js`) to capture user input and display it in the output div when the button is clicked.
4. Implement color-changing functionality using event listeners on color buttons to dynamically modify the page background.
5. Add counter logic with increment and decrement buttons that update a displayed counter value in real time.

Result

The interactive webpage was successfully created with text display, background color changes, and counter functionalities.

All JavaScript event-driven interactions were tested and confirmed to be working as expected.

Exercise 1c: Simple User Registration Form

Aim

To create a user registration form using HTML, CSS, and JavaScript that validates user input and displays the submitted details.

Procedure

1. Design the HTML form with fields for Full Name, Email, Password, Date of Birth, Age, Country (dropdown), and a Terms & Conditions checkbox.
2. Style the form using CSS with a centered card layout, proper spacing, and a clean visual appearance.
3. Implement JavaScript to auto-calculate the user's age when the Date of Birth field is changed.
4. Add form validation logic to ensure all required fields are filled and the Terms checkbox is checked before submission.
5. On successful submission, display a confirmation message and render the submitted details in a styled card below the form.

Result

The user registration form was successfully created with auto-age calculation and form validation features.

Upon valid submission, the registered details are displayed in a summary card confirming successful registration.

Exercise 1d: Interest Form using HTML, CSS and JavaScript

Aim

To design an interest selection form using HTML, CSS, and JavaScript that allows users to select their preferences via checkboxes and dropdowns.

Procedure

1. Create the HTML structure with a form containing checkboxes for interests (Sports, Music, Reading, etc.) and dropdown menus for education level and programming language.
2. Style the form using CSS to present a clean, centered card layout with properly spaced checkboxes and select elements.
3. Write JavaScript to handle form submission by collecting all checked interest values into an array.

4. Validate that at least one interest is selected and both dropdown values are chosen before displaying results.
5. Display the selected information (interests, education level, and programming language) in a styled result card below the form.

Result

The interest form was successfully created with checkbox selections and dropdown menus for user preferences.

Upon submission, the selected interests and choices are validated and displayed in a summary card below the form.

Exercise 2: React Frontend Fundamentals and Form

Exercise 2a: Interactive Aadhaar Card Viewer

Aim

To build an interactive Aadhaar card viewer web application using React that takes user input and renders a styled Aadhaar card component.

Procedure

1. Set up a standalone HTML file with React 18 and Babel loaded via CDN links for in-browser JSX compilation.
2. Create an input form component with fields for Name, DOB, Gender, Address, and Aadhaar number using React `useState` hook.
3. Build an `AadhaarCard` component that displays the entered details in a card layout resembling an official Aadhaar card with header, photo placeholder, and formatted number.
4. Implement form validation to ensure all fields are filled and the Aadhaar number is exactly 12 digits before generating the card.
5. On clicking "Generate Aadhaar Card", pass the form data as props to the `AadhaarCard` component which renders below the form.

Result

The Aadhaar card viewer application was successfully built using React with CDN, rendering a styled card on valid form submission.

The application demonstrates React component composition, state management with hooks, and prop-based data flow.

Exercise 2b: User Registration Form (React)

Aim

To create a user registration form using React with form validation, state management using hooks, and conditional rendering of submitted details.

Procedure

1. Set up a standalone HTML file with React 18 and Babel CDN for in-browser JSX compilation.
2. Create a functional component with `useState` hooks to manage form fields (name, email, password, phone, gender, city) and validation errors.
3. Implement a `validate` function that checks for empty fields, valid email format, minimum password length, and 10-digit phone number.
4. On form submission, run validation and either display error messages below each invalid field or show a success message.
5. Use conditional rendering to display a "Registration Details" card below the form only when all fields pass validation.

Result

The React registration form was successfully created with real-time validation and conditional rendering of user details.

The application demonstrates React hooks (`useState`), controlled components, and form handling patterns.

Exercise 2c: Counter App (React)

Aim

To develop a counter application using React that demonstrates state management with the `useState` hook, event handling, and dynamic styling.

Procedure

1. Set up a standalone HTML file with React 18 and Babel CDN for in-browser JSX compilation with a dark-themed UI.
2. Create a functional component with `useState` hooks to manage the counter value and a configurable step size.
3. Implement increment, decrement, and reset buttons that modify the counter state by the step value.
4. Add dynamic color styling to the counter display — green for positive, red for negative, and white for zero values.
5. Provide a step size input field that allows the user to customize the increment/decrement value for flexible counting.

Result

The React counter application was successfully created with increment, decrement, reset, and configurable step size features.

The counter dynamically changes color based on its value, demonstrating React state management and conditional styling.

Exercise 3: Advanced React State Management

Exercise 3a: Theme Toggle using React Context

Aim

To develop a React application with a theme toggle switch that switches between light and dark modes using React Context API for global state management.

Procedure

1. Set up a standalone HTML file with React 18 and Babel CDN, and define CSS classes for both light and dark themes.

2. Create a `ThemeContext` using `React.createContext()` and a `ThemeProvider` component that manages the theme state with `useState`.
3. Implement a `toggleTheme` function in the provider that switches between 'light' and 'dark' and updates the document body class.
4. Build a `Navbar` component that consumes the context via `useContext` and renders a toggle button displaying the current mode.
5. Create a `Content` component that reads the theme from context and displays themed cards with information about how the Context API works.

Result

The theme toggle application was successfully built using React Context API, enabling seamless switching between light and dark modes. All child components correctly consume the global theme state without prop drilling, demonstrating effective use of `createContext` and `useContext` hooks.

Exercise 4: Backend API Development with Express

Exercise 4a: Health Check Route

Aim

To create a simple Express.js server with a health-check route that returns a 200 OK status and a JSON message confirming the server is running.

Procedure

1. Initialize a Node.js project and install Express.js as a dependency using `npm install express`.
2. Create the server file (`app.js`) and import Express to set up a basic HTTP server.
3. Define a GET route at `/api/health` that responds with a 200 status code and JSON body `{ "status": "Server is running smoothly" }`.
4. Configure the server to listen on port 3000 and log a confirmation message with the server URL.

5. Test the route by running `node app.js` and accessing `http://localhost:3000/api/health` in a browser or using `curl`.

Result

The Express server was successfully created with a health-check endpoint at GET `/api/health` returning a 200 OK response.

The JSON response `{ "status": "Server is running smoothly" }` confirms the server is operational and ready to handle requests.

Exercise 4b: Medical Scan Management System (CRUD)

Aim

To build a backend API for a medical scan management system using Express.js with in-memory CRUD operations for patient appointment bookings.

Procedure

1. Set up an Express server with `express.json()` middleware to parse JSON request bodies, and initialize an in-memory array for storing appointments.
2. Create a POST `/api/scans` route that accepts `patientName`, `scanType`, and `date` from the request body, assigns a unique ID, and returns the created object with a 201 status.
3. Create a GET `/api/scans` route that returns the complete list of all scheduled scan appointments as a JSON array.
4. Create a PUT `/api/scans/:id` route that finds an appointment by ID and updates its details; returns a 404 error if the ID does not exist.
5. Create a DELETE `/api/scans/:id` route that removes a specific appointment from the array and returns a success confirmation message.

Result

The medical scan management API was successfully built with full CRUD operations (Create, Read, Update, Delete) using an in-memory array.

All endpoints were tested and verified to handle valid requests and return appropriate error responses for invalid IDs.

Exercise 4c: Request Logger Middleware

Aim

To create a custom Express middleware that logs every incoming request with timestamp, HTTP method, and URL path for auditing purposes.

Procedure

1. Set up an Express server and define a custom middleware function named `requestLogger`.
2. Inside the middleware, capture the current date/time using `new Date().toISOString()`, the HTTP method (`req.method`), and the URL path (`req.url`).
3. Log the captured information to the console in the format `[timestamp] METHOD /path` (e.g., `[2026-02-19T10:00:00Z] POST /api/scans`).
4. Call the `next()` function at the end of the middleware to pass the request to the actual route handlers.
5. Apply the middleware globally using `app.use(requestLogger)` so it intercepts every incoming request before it reaches any route.

Result

The request logger middleware was successfully created and applied globally to intercept all incoming HTTP requests.

Each request is logged to the console with its timestamp, method, and URL path, enabling effective server-side auditing.

Exercise 4d: Validation Middleware

Aim

To create an Express middleware that validates incoming user registration data (email and password) before it reaches the route handler.

Procedure

1. Set up an Express server with JSON body parsing enabled using `express.json()`.

2. Create a middleware function named `validateUser` that extracts `email` and `password` from `req.body`.
3. Inside the middleware, check if the email contains an "@" symbol; if not, return a 400 Bad Request with a specific error message.
4. Check if the password is at least 6 characters long; if not, return a 400 Bad Request with an appropriate error message.
5. If both validations pass, call `next()` to proceed to the POST `/register` route handler which responds with "User Registered Successfully".

Result

The validation middleware was successfully created to validate email format and password length before processing registration.

Invalid requests receive a 400 Bad Request with specific error messages, while valid requests proceed to register the user successfully.

Exercise 5: MongoDB, Mongoose & JWT Authentication

Exercise 5a: MongoDB Connection with Express and Mongoose

Aim

To establish a connection between a Node.js Express server and a MongoDB Atlas cloud database named `patient_scans_db` using Mongoose.

Procedure

1. Initialize a Node.js project and install `express` and `mongoose` as dependencies using npm.
2. Import Express and Mongoose in `app.js` and define the MongoDB Atlas connection URI with the database name.
3. Use `mongoose.connect(MONGO_URI)` to establish the connection, handling success and error cases with `.then()` and `.catch()`.

4. On successful connection, log a confirmation message and start the Express server on port 3000.
5. Add a root GET route that returns a JSON message confirming the database connection, and test by running `node app.js`.

Result

The Express server was successfully connected to MongoDB Atlas cloud database (patient_scans_db) using Mongoose.

The connection was verified with a confirmation message in the console and a JSON response on the root route.

Exercise 5b: Mongoose Scan Schema

Aim

To define a Mongoose schema for a Scan document with required fields, default values, and create Express routes to add and retrieve scan records from MongoDB.

Procedure

1. Set up an Express server with Mongoose connected to MongoDB Atlas (patient_scans_db) and enable JSON body parsing.
2. Define a Mongoose schema for `Scan` with fields: `patientName` (String, required), `scanType` (String, required), `scanDate` (Date, default: current date), and `resultStatus` (String, default: 'Pending').
3. Create a Mongoose model from the schema using `mongoose.model('Scan', scanSchema)`.
4. Implement a POST `/api/scans` route that creates and saves a new Scan document from the request body, returning a 201 status on success.
5. Implement a GET `/api/scans` route that retrieves all scan documents from the database and returns them as a JSON array.

Result

The Mongoose Scan schema was successfully defined with required validations and default values for scanDate and resultStatus.

Scan documents can be created via POST and retrieved via GET, with data persisting in the MongoDB Atlas cloud database.

Exercise 5c: Trainee Schema with Validation

Aim

To define a Mongoose schema for a Trainee document with robust validation rules and create an Express POST route that handles validation errors gracefully.

Procedure

1. Set up an Express server connected to MongoDB Atlas (training_institute_db) with JSON body parsing enabled.
2. Define a Trainee schema with fields: `fullName` (String, required), `email` (String, required, unique), `courseRegistered` (String, required), and `hasCSEBackground` (Boolean, default: false).
3. Create an Express POST route at `/api/trainees` that attempts to save the request body as a new Trainee document.
4. Implement error handling in the catch block to detect Mongoose `ValidationError` — extract specific error messages and return a 400 Bad Request response.
5. Handle duplicate email errors (MongoDB error code 11000) separately, returning a clear “Email already exists” message with 400 status.

Result

The Trainee schema was successfully defined with required field validations and unique email constraint enforced by MongoDB.

Validation errors are caught and returned as descriptive 400 Bad Request responses, ensuring robust data integrity.

Exercise 5d: Course Filter Route

Aim

To create an Express GET route that filters and returns trainees enrolled in a specific course using URL parameters and MongoDB queries.

Procedure

1. Set up an Express server connected to MongoDB Atlas (training_institute_db) with the Trainee schema defined.
2. Create a GET route at `/api/trainees/course/:courseName` that extracts the course name from `req.params.courseName`.
3. Use `Trainee.find({ courseRegistered: courseName })` to query the database for trainees matching the specified course.
4. Return the filtered results as a JSON array, so querying `/api/trainees/course/React` returns only React students.
5. Test the route by first adding trainees via POST and then filtering by different course names to verify correct results.

Result

The course filter route was successfully created, returning only trainees enrolled in the specified course from the URL parameter.

The route correctly queries MongoDB using the `courseName` parameter and returns an empty array if no trainees match the filter.

Exercise 5e: JWT Authentication

Aim

To secure an API endpoint using JSON Web Tokens (JWT) so that only authenticated administrators can access the trainees list, with a React frontend to handle login and token-based requests.

Procedure

1. Set up an Express server with Mongoose, JWT (`jsonwebtoken`), and CORS enabled, connected to MongoDB Atlas (training_institute_db).

2. Create a POST `/api/login` route that validates admin credentials and returns a signed JWT token with a 1-hour expiry.
3. Create an `authenticateToken` middleware that extracts the Bearer token from the Authorization header, verifies it using `jwt.verify()`, and blocks access with 401/403 errors if the token is missing or invalid.
4. Apply the `authenticateToken` middleware to the GET `/api/trainees` route to protect it so only logged-in admins can access the trainee list.
5. Build a React frontend (using CDN) with a login form that stores the JWT token, then uses it in the Authorization header when fetching the protected trainees endpoint.

Result

The API endpoint was successfully secured using JWT authentication, blocking unauthorized access to the trainees list.

The React frontend correctly handles login, stores the token, and passes it as a Bearer token to access the protected route.

Exercise 6: Deployment

Exercise 6a: Deploying React Project to Vercel

Aim

To deploy a completed React frontend application to Vercel cloud platform, making it accessible over the public internet.

Procedure

1. Install Vercel CLI globally using `npm install -g vercel` and log in to your Vercel account using `vercel login`.
2. Navigate to the project directory containing the React application (e.g., the JWT Auth frontend from Exercise 5e).
3. Run `vercel` in the terminal, follow the prompts to link the project to your Vercel account, and configure the build settings.

4. Vercel automatically detects the project type, builds it, and deploys to a unique URL (e.g., `https://your-project.vercel.app`).
5. Verify the deployment by opening the provided URL in a browser to confirm the React application loads and functions correctly.

Result

The React application was successfully deployed to Vercel and is accessible via a public URL over the internet.

The deployment was verified by accessing the live URL and confirming all features function correctly in the cloud environment.
